236. Lowest Common Ancestor of a Binary Tree Medium

Acceptance: 68.5% | LeetCode

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

```
**Input:** root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
**Output:** 3
**Explanation:** The LCA of nodes 5 and 1 is 3.
```

Example 2:

```
**Input:** root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
**Output:** 5
**Explanation:** The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.
```

Example 3:

```
**Input:** root = [1,2], p = 1, q = 2
**Output:** 1
```

Constraints:

- The number of nodes in the tree is in the range `[2, 105]`.
- `-109 <= Node.val <= 109`
- All `Node.val` are **unique**.
- `p != q`
- `p` and `q` will exist in the tree.

Claude Opus Solution

Lowest Common Ancestor of a Binary Tree

1. Intuition

The key insight is to think about **where** `p` and `q` can be located relative to any node:

- If `p` and `q` are found in **different subtrees** of a node, then that node **must be** their LCA (it's the "split point").
- If both are in the **same subtree**, the LCA is deeper in that subtree.
- A node can be a descendant of itself, so if the current node **is** `p` or `q`, we can stop and return it.

This naturally maps to a **post-order DFS**: we explore both subtrees first, then decide based on what each subtree returns. The first node where the search results "meet" from both sides is the LCA.

2. Approach (Pseudocode)

```
function LCA(root, p, q):
    # Base case 1: reached null → nothing found here
    if root is None:
        return None

    # Base case 2: found one of the targets → return it upward
    if root == p or root == q:
        return root

    # Recurse into both subtrees
    left  = LCA(root.left, p, q)
    right = LCA(root.right, p, q)

    # If both subtrees returned a non-null node,
    # then p and q are split across them → current node is LCA
    if left and right:
        return root

    # Otherwise, propagate whichever side found something (or None)
    return left or right
```

3. Python Solution

```python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # Base case 1: We've fallen off the tree (past a leaf).
        # Nothing to find here, so return None.
        if root is None:
            return None

        # Base case 2: The current node is one of the targets.
        # We return it immediately. This handles the "a node can be
        # a descendant of itself" rule — if the other target lies below,
        # this node is the LCA; otherwise the parent will combine results.
        if root is p or root is q:
            return root

        # Recurse on both subtrees.
        # Each call returns:
        #   - a target node (p or q) if found in that subtree, OR
        #   - the LCA if both targets were found in that subtree, OR
        #   - None if neither target is in that subtree.
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        # If BOTH sides returned something non-null, it means p and q
        # were found in different subtrees of the current node.
        # Therefore, the current node is the lowest common ancestor.
        if left and right:
            return root

        # Otherwise, exactly one side (or neither) found something.
        # Propagate the non-None result upward (or None if both are None).
        # - If only left found a node, that node is either a target
        #   or an already-determined LCA; pass it up.
        # - Same logic for right.
        return left or right
```

> **Note:** I use `is` instead of `==` for node identity comparison since we're comparing object references (the problem guarantees `p` and `q` are actual nodes in the tree). This is cleaner and avoids relying on `__eq__`.

---

4. Complexity Analysis

Time Complexity: O(N) - We visit each node in the tree **at most once** in the worst case (e.g., when `p` and `q` are deep in opposite subtrees, or one of them doesn't share an early ancestor). - Each visit does O(1) work, so total is O(N) where N is the number of nodes.

Space Complexity: O(H) where H is the height of the tree - The recursion stack consumes space proportional to the depth of recursion. - **Best/Average case (balanced tree):** H = O(log N). - **Worst case (skewed tree, e.g., a linked list):** H = O(N).

> The README claims O(1) space, but this is **incorrect** — recursion always uses O(H) stack space. Only an iterative approach with parent pointers and no recursion could potentially reduce this, but it would still typically use O(N) for storing parent maps.

---

5. Alternative Approaches

A. Iterative with Parent Pointers (BFS/DFS + Set)

1. Traverse the tree, building a `parent` map (`child → parent`) until both `p` and `q` are found.

2. Walk up from `p`, collecting all ancestors into a set.
3. Walk up from `q`; the first ancestor already in the set is the LCA.

```python
def lowestCommonAncestor(self, root, p, q):
    parent = {root: None}
    stack = [root]
    # Build parent links until both targets are discovered
    while p not in parent or q not in parent:
        node = stack.pop()
        if node.left:
            parent[node.left] = node
            stack.append(node.left)
        if node.right:
            parent[node.right] = node
            stack.append(node.right)
    # Collect all ancestors of p
    ancestors = set()
    while p:
        ancestors.add(p)
        p = parent[p]
    # First ancestor of q that's also an ancestor of p
    while q not in ancestors:
        q = parent[q]
    return q
```

- **Time:** O(N), **Space:** O(N). Avoids deep recursion (good for very tall trees).

B. Path-Finding Approach (the `my_solution.py`)

- Find the full root-to-`p` and root-to-`q` paths, then find the last common node.
- Correct but uses **extra O(N) space** for paths and copies lists during recursion (inefficient). The recommended recursive solution is strictly better.

C. Binary Search Tree Variant

- If the tree were a **BST** (LeetCode 235), you could use the BST property to walk down comparing values — O(H) time, O(1) space. But that doesn't apply here since this is a general binary tree.

Recommended solution: The recursive post-order DFS (Solution above) — it's clean, optimal in time, and elegant. The "split point" pattern is a fundamental tree technique worth internalizing.

Codex Solution

1. Intuition

For each subtree, ask:

> "Does this subtree contain `p`, `q`, or their LCA?"

A recursive DFS can return one of three things:

1. `None`
   → this subtree contains neither `p` nor `q`.
2. `p` or `q`
   → this subtree contains one of the target nodes.
3. The LCA node
   → this subtree already contains both `p` and `q`.

The key insight:

- If `p` is found in the left subtree and `q` is found in the right subtree, then the current node is their lowest common ancestor.
- If the current node itself is `p` or `q`, then it can be the LCA because a node is allowed to be a descendant of itself.

---

2. Approach

Pseudocode

```
function LCA(root, p, q):
    if root is null:
        return null

    if root is p or root is q:
        return root

    left_result = LCA(root.left, p, q)
    right_result = LCA(root.right, p, q)

    if left_result is not null and right_result is not null:
        return root

    if left_result is not null:
        return left_result

    return right_result
```

Meaning of the recursive return value

For a node `root`:

- Search its left subtree.
- Search its right subtree.
- If both sides return non-null values, then one target was found on each side, so `root` is the LCA.
- If only one side returns a non-null value, pass that result upward.
- If neither side finds anything, return `None`.

---

3. Python Solution

```python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

import sys

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # The tree can have up to 10^5 nodes.
        # In a skewed tree, recursion depth can also be 10^5.
        sys.setrecursionlimit(200000)

        def dfs(node):
            """
            Returns:
                None if this subtree contains neither p nor q.
                p or q if this subtree contains one of them.
                The LCA node if this subtree contains both.
            """

            # Base case: empty subtree contains nothing.
            if node is None:
                return None

            # If the current node is p or q, return it immediately.
            # This also handles the case where one node is an ancestor of the other.
            if node is p or node is q:
                return node

            # Search left subtree.
            left = dfs(node.left)

            # Search right subtree.
```

```
            right = dfs(node.right)

            # If both sides found something, then p and q are split
            # between the left and right subtrees.
            # Therefore, current node is their lowest common ancestor.
            if left is not None and right is not None:
                return node

            # Otherwise, return whichever side found a target node or LCA.
            # If neither side found anything, this returns None.
            return left if left is not None else right

        return dfs(root)
```

---

4. Complexity Analysis

Let `n` be the number of nodes in the tree.

Time Complexity: `O(n)`

In the worst case, we may visit every node exactly once.

For example, if `p` and `q` are deep in different parts of the tree, DFS needs to explore most or all nodes.

Space Complexity: `O(h)`

Where `h` is the height of the tree.

This space is used by the recursion call stack.

- Balanced tree: `O(log n)`
- Skewed tree: `O(n)`

So worst-case space complexity is:

```
O(n)
```

---

5. Alternative Approaches

1. Store Root-to-Node Paths

Find the path from `root` to `p`, and the path from `root` to `q`.

Example:

```
path_to_p = [3, 5]
path_to_q = [3, 1]
```

Then compare the two paths from the beginning. The last common node is the LCA.

Complexity:

- Time: `O(n)`
- Space: `O(h)` for each path, or `O(n)` worst case

This is easy to understand but less elegant than the recursive DFS solution.

---

2. Parent Pointer Hash Map

Use DFS or BFS to store each node's parent.

Then:

1. Walk upward from `p`, storing all ancestors in a set.
2. Walk upward from `q` until you find a node already in that set.

That node is the LCA.

Complexity:

- Time: `O(n)`
- Space: `O(n)`

This approach is useful if the tree nodes have no parent pointers and you want an iterative solution.

---

3. Tarjan's Offline LCA Algorithm

If there are many LCA queries on the same tree, Tarjan's algorithm can answer them efficiently using Union-Find. However, for a single query like this problem, the recursive DFS solution is simpler and optimal.